

Partie III : Tests et Validation du Système

Introduction

Cette partie présente les procédures de test et de validation du système multi-agents développé pour le traitement des données oncologiques. L'objectif est de décrire la méthodologie de test sans présenter de résultats expérimentaux ou de prétentions non vérifiées.

0.1 Générateur de Données de Test

Pour évaluer les fonctionnalités du système, un générateur de données de test a été implémenté. Ce module permet de créer des jeux de données synthétiques représentatifs des différentes spécialités oncologiques.

0.1.1 Structure du Générateur

Le générateur est implémenté sous forme d'une classe Python contenant des méthodes statiques pour chaque type de données.

```
1 class TestDataGenerator:
2     """
3     G n rateur de donn es de test pour la validation du syst me
4     Cette classe produit des donn es synth tiques pour tester
5     les diff rentes fonctionnalit s du syst me multi-agents.
6     """
7
8     @staticmethod
9     def generate_clinical_oncology_data():
10         """G n re des donn es simul es d'oncologie clinique"""
11         return {
12             'patient_id': 'PAT001',
13             'age': 58,
14             'gender': 'female',
15             'cancer_type': 'breast_cancer',
16             'stage': 'IIIA'
17         }
18
19     @staticmethod
20     def generate_molecular_biology_data():
21         """G n re des donn es simul es de biologie mol culaire
22         """
23         return {
24             'patient_id': 'PAT002',
25             'mutations': [
26                 {'gene': 'EGFR', 'type': 'L858R'},
27                 {'gene': 'TP53', 'type': 'R175H'}
28             ]
29         }
30
31     @staticmethod
32     def generate_pathology_data():
33         """G n re des donn es simul es de pathologie"""
```

```

33         return {
34             'patient_id': 'PAT003',
35             'pathology_report': 'Invasive ductal carcinoma'
36         }
37
38     @staticmethod
39     def generate_imaging_data():
40         """G n re des donn es simul es d'imagerie m dicale"""
41         return {
42             'patient_id': 'PAT004',
43             'imaging_modality': 'CT_scan'
44         }
45
46     @staticmethod
47     def generate_endocrinology_microbiome_data():
48         """G n re des donn es simul es endocriniennes et
49             microbiomiques"""
50         return {
51             'patient_id': 'PAT005',
52             'hormone_levels': {'testosterone': 850}
53         }
54
55     @staticmethod
56     def generate_epidemiology_data():
57         """G n re des donn es simul es pidmiologiques """
58         return {
59             'patient_id': 'PAT006',
60             'risk_factors': {'family_history': True}
61         }

```

Listing 1 – Structure de la classe TestDataGenerator

0.1.2 Description des Données Générées

Les données générées par cette classe sont purement illustratives et servent uniquement à démontrer la structure d'entrée attendue par le système. Chaque méthode retourne un dictionnaire Python structuré selon les besoins de l'agent spécialisé correspondant.

0.2 Procédures de Test

0.2.1 Test Unitaires des Agents

Chaque agent fait l'objet de tests unitaires pour vérifier sa capacité à traiter les données d'entrée et à générer des annotations appropriées.

```

1 def test_clinical_oncology_agent():
2     """Test de l'agent d'oncologie clinique"""
3     agent = ClinicalOncologyAgent()
4     test_data = TestDataGenerator.generate_clinical_oncology_data()
5
6     # V rification de la capacit annoter
7     can_annotate = agent.can_annotate(test_data)

```

```

8
9     # G n r a t i o n d' a n n o t a t i o n s
10    if can_annotate:
11        annotation = agent.annotate(test_data)
12        return annotation
13    return None

```

Listing 2 – Exemple de test unitaire pour un agent

0.2.2 Test d'Intégration

Les tests d'intégration vérifient l'interaction entre les différents composants du système.

```

1 def test_system_integration():
2     """Test d'int gration du syst me complet"""
3     # Initialisation du syst me
4     core = CancerDataIntelligenceCore()
5
6     # G n r a t i o n d e d o n n e s d e t e s t
7     test_data = TestDataGenerator.generate_multi_specialty_data()
8
9     # Traitement des donn es
10    result = core.process_data(test_data)
11
12    # V r i f i c a t i o n d e l a s t r u c t u r e d e s o r t i e
13    expected_keys = ['status', 'classification', 'annotations']
14    return all(key in result for key in expected_keys)

```

Listing 3 – Test d'intégration du système

0.3 Validation de la Qualité des Annotations

Le système intègre des mécanismes de validation interne pour assurer la qualité des annotations générées.

0.3.1 Vérification de la Cohérence

Chaque annotation est accompagnée d'un score de confiance et d'une liste de preuves justifiant les décisions prises par l'agent.

0.3.2 Validation Croisée entre Agents

Lorsque plusieurs agents sont impliqués dans le traitement des mêmes données, le système compare leurs annotations et recherche les éventuelles incohérences.

0.4 Scénarios de Test

0.4.1 Scénario 1 : Données Monospécialité

Test du système avec des données relevant d'une seule spécialité oncologique.

0.4.2 Scénario 2 : Données Multispécialités

Test du système avec des données complexes impliquant plusieurs spécialités simultanément.

0.4.3 Scénario 3 : Données Incomplètes

Évaluation de la robustesse du système face à des données manquantes ou incomplètes.

0.5 Exécution des Tests

Le système fournit un script d'exécution principale permettant de lancer l'ensemble des procédures de test.

```
1 def main():
2     """Fonction principale pour ex cuter les tests"""
3     print("D marriage des proc dures de test...")
4
5     # Initialisation
6     test_results = {}
7
8     # Ex cution des tests unitaires
9     test_results['unit_tests'] = run_unit_tests()
10
11    # Ex cution des tests d'int gration
12    test_results['integration_tests'] = run_integration_tests()
13
14    # Ex cution des sc narios de test
15    test_results['scenarios'] = run_test_scenarios()
16
17    print("Proc dures de test termin es.")
18    return test_results
19
20 if __name__ == "__main__":
21     main()
```

Listing 4 – Script principal d'exécution des tests

0.6 Conclusion

Cette section a décrit les procédures de test et de validation du système multi-agents. Les tests sont conçus pour vérifier la fonctionnalité de chaque composant individuellement et leur intégration dans le système complet. La méthodologie de test suit les bonnes pratiques du génie logiciel et assure la robustesse du système pour le traitement des données oncologiques.